

Guía del Profesor — Tema 07: Paradigma Lógico — Clase 2+3

Para el docente: documento autocontenido para repaso express del tema. Contiene plan de clase, rutas a artefactos, extractos clave de fuentes, banco de preguntas y FAQ anticipado.

Materia: Paradigmas y Lenguajes de Programación 2026 **Docente titular:** Matías Gel

Duración: 240 min (clase doble — fusión clases 2+3 módulo III) **Generada:** 2026-04-21

1. Resumen ejecutivo (60 segundos)

- **Qué:** segunda y tercera parte de Prolog en una clase doble.
 - **Cómo:** 13 bloques + descanso a los 120 min; 158 filminas; 1 ejercicio colaborativo.
 - **Dónde están los recursos:** todo en `salida/cursadas/2026/temas/07-paradigma-logico-avanzado/`.
 - **Qué NO sacrificar si te atrasás:** B1 (unificación), B2 (SLD), B3 (backtracking), B8 (listas), B9 (acumulador), cierre.
-

2. Índice de artefactos

Artefacto	Ruta	Para qué
Diseño	<code>diseno.md</code>	Bloques, objetivos, scope
Minuta	<code>minuta.md</code>	Guion operativo minuto a minuto
Filminas	<code>filminas.md</code>	158 slides Markdown con <code>@tipo:</code>
Guía de estudio	<code>guia-estudio.md</code>	Para los alumnos (rica)
TP	<code>tp.md</code>	Consigna desarrollo
Plan Slides	<code>slides/plan-filminas-07-paradigma-logico-avanzado.json</code>	Plan JSON v3
Slides publicadas	<code>slides/slides-url.txt</code>	URL Google Slides
Assets	<code>slides/assets/</code>	Imágenes generadas

3. Plan de clase (240 min)

Min	Bloque	Contenido	Pedagogía
0–10	B0	Repaso Clase 1	Quiz relámpago
10–35	B1	Unificación (algoritmo + MGU + occurs-check)	Exposición + pizarrón
35–60	B2	Resolución SLD + árbol SLD	Pizarrón
60–85	B3	Backtracking + choice points + trail	Live SWISH
85–105	B4	Corte (!): verde vs. rojo, (-> ;)	Live + dilema
105–120	B5	Negación por falla, CWA, dif/2	Contra-ejemplos
120–130	 DESCANSO	10 min	Obligatorio
130–150	B6	Ejercicio colaborativo (3 ejercicios en pares)	Trabajo en pares
150–170	B7	Aritmética: is/2 , operadores, anti-patrones	Live-coding
170–195	B8	Listas: [H\ T] , member , append , length	Derivación en vivo
195–220	B9	Recursión con acumulador + LCO	Comparación
200–220	B10	Meta-predicados: findall , bagof , setof	Live-coding
220–240	B11	Aplicaciones (coloreo, vuelos, Sudoku, DCG)	Demos breves
240–250	B12	Prolog 2026 (Datalog, neuro-simbólico)	Exposición (compresible)

Min	Bloque	Contenido	Pedagogía
250–260	B13	Cierre + TP + pregunta de salida	Cierre activo

Nota: los 10 min extra (hasta 260) son provisión para preguntas/pausas.

4. Extractos clave de las fuentes

De Sebesta (cap. 16)

"Logic programs are sets of statements that represent facts and rules about some problem. An execution of a logic program is a proof that a goal statement follows from the program statements."

"Prolog has two basic statement forms; these correspond to the headless and headed Horn clauses of predicate calculus."

"Full unification requires the occurs check, but most Prolog systems omit it for efficiency."

"The cut operator is used to eliminate backtracking choice points and improve efficiency, but at the cost of declarative transparency."

De Gabbrielli & Martini (cap. Logic Programming)

"Logic programming realizes Kowalski's slogan: 'Algorithm = Logic + Control'. The programmer provides the logic; the system (and sometimes the programmer with cuts and orderings) provides the control."

"Unification is the single information-transfer mechanism in pure logic programming. No assignment, no parameter passing, no return value."

"The SLD resolution is complete for definite Horn clauses: if a goal has a proof, SLD will find it (provided the search strategy is fair)."

De Louden & Lambert (cap. 4)

"Prolog's list notation `[H|T]` is syntactic sugar over the binary constructor `. / 2`. This cons-list structure enables natural pattern matching on recursive structures."

De Sterling & Shapiro (*The Art of Prolog*, cap. 3)

"Recursive programming in Prolog has two canonical forms: direct recursion and accumulator recursion. The latter enables last-call optimization — the key to efficient logic programming."

De Triska (*The Power of Prolog*, 2020)

"Use `dif/2` instead of `\+` whenever you mean 'these terms are different'. `\+` is a meta-logical cop-out that works only when the goal is fully ground."

5. Banco de preguntas para clase

B1 — Unificación

- ¿Cuál es la MGU de `f(X, Y)` con `f(a, X)` ?
- ¿Por qué `?- X = f(X).` es un bug silencioso en SWI por default?
- Diferencia entre `=`, `==`, `=..`.

B2 — SLD

- ¿Qué significa "leftmost + top-down"?
- ¿Qué pasa si invierto las dos cláusulas de `ancestro/2` ?

B3 — Backtracking

- ¿Cuándo se crea un choice point?
- ¿Cómo revierte Prolog las ligaduras?

B4 — Corte

- ¿Corte verde o rojo en `max/3` con `X >= Y, !` ?
- ¿Qué pasa si quito el `!` de `abs/2` ?

B5 — Negación

- ¿Por qué `?- \+ X = 1, X = 2.` falla?
- ¿Cuándo usar `dif/2` en lugar de `\+` ?

B7 — Aritmética

- ¿Qué imprime `?- X = 2+3, X == 5.` ?
- ¿Por qué `?- 5 is X + 3.` da error?

B8 — Listas

- Escribir `member/2` sin mirar.
- ¿`append/3` es reversible? Dar 3 usos distintos.

B9 — Recursión

- Convertir `suma([], 0). suma([H|T], S) :- suma(T, ST), S is H + ST.` en acumulador.
- ¿Qué es LCO?

B10 — Meta-predicados

- Diferencia entre `findall`, `bagof`, `setof`.
- ¿Qué hace `^` en `setof`?

6. FAQ anticipado

“¿Por qué en SWI `?- X = f(X).` da éxito?”

Porque SWI-Prolog desactiva el occurs-check por default (costo lineal). Activar con `set_prolog_flag(occurs_check, true).` o usar `unify_with_occurs_check/2`.

“¿Cuándo usar `!` y cuándo `(-> ;)`?”

Preferir `(-> ;)`. Usar `!` solo cuando: - Sea corte **verde** (optimización) - O cuando la sintaxis `(-> ;)` no sea clara

Documentar siempre los cortes rojos con comentario.

“¿Prolog se usa en la industria?”

Sí, en nichos: - **SWI-Prolog**: sistemas clínicos, análisis estático - **Datalog** (derivado): Google Logica, Soufflé, Meta - **Tau Prolog**: JavaScript/navegador - **Neuro-simbólico**: DeepProbLog, AlphaProof 2024

“¿Por qué `?- X is Y + 1.` da error si Y está libre?”

Porque `is/2` requiere que el lado derecho sea **ground** (sin variables libres). Para aritmética reversible usar `library(clpfd)` con `#=`.

“¿Por qué `append/3` es ‘especial’?”

Porque relaciona tres listas en lugar de “computar”. Podés fijar cualquier combinación y Prolog completa las demás. Es el ejemplo más nítido de **declaratividad**.

“¿Qué hace `between/3`?”

Genera (o verifica) enteros en un rango. Ejemplo:

```
?- between(1, 5, X).  
X = 1 ; X = 2 ; ... ; X = 5.
```

“¿`\+` es como `!` en Haskell?”

NO. `\+` es **falla demostrativa** (CWA). Haskell no tiene negación lógica — tiene `not :: Bool -> Bool` que es diferente.

“¿`findall` con goal vacío?”

```
?- findall(X, false, L). → L = [] . Nunca falla. Es su signature.
```

“¿Cómo debugueo en Prolog?”

- `?- trace, goal.` → traza paso a paso
- `?- spy(predicado/aridad).` → breakpoint en predicado específico
- `?- notrace.` → sale del modo debug

7. Checklist pre-clase (día anterior)

- ☐ SWI-Prolog actualizado (versión 9.x)
 - ☐ SWISH probado con conexión (<https://swish.swi-prolog.org/>)
 - ☐ `familia.pl`, `colores.pl`, `vuelos.pl`, `nreinas.pl` preparados en una carpeta accesible
 - ☐ Filminas publicadas en Google Slides (URL en `slides/slides-url.txt`)
 - ☐ Pizarrón limpio, 2 fibras distintos colores
 - ☐ Cronómetro o celular para los 10 min de descanso
 - ☐ Post-its para la pregunta de salida
 - ☐ Enlazar esta guía en Moodle / Classroom
-

8. Cómo evaluar el aprendizaje durante la clase

Pulso rápido (cada 25 min)

Pregunta sí/no a mano alzada: - ¿Lo entendieron? (pulgar arriba/abajo/al medio) - Si más de 30% al medio → 2 min de repaso del último bloque antes de avanzar

Pulso profundo (al final de cada gran bloque: B3, B5, B7, B9)

1 pregunta abierta que pida **explicación** (no solo respuesta). Ejemplos: - “Explicá en voz alta cómo se crea un choice point” - “¿Por qué preferimos acumulador sobre recursión directa?”

Pulso final (salida)

Pregunta de salida: “Dame un ejemplo cotidiano que sería más fácil en Prolog que en Python” — escribir en post-it.

Métrica: si al menos 70% escribe algo **coherente**, la clase funcionó.

9. Ajustes según audiencia

Si la clase es fuerte:

- Extender B11 (aplicaciones) con el código completo de N-reinas CLP(FD).
- En B12, mostrar un paper de DeepProbLog.
- Dejar tiempo extra para preguntas abiertas.

Si la clase es débil:

- Comprimir B10 (meta-predicados) a solo `findall/3`.
- Saltar F-131 (agrupación con variable libre — es avanzado).
- Agregar 5 min extra en B1 (unificación) — es la base.

Si hay alumnos repitiendo:

- Asignarles el rol de “asistente” — que ayuden en B6 a sus pares.
-

10. Retroalimentación con iteraciones previas

De Tema 06 (clase 1): - Los alumnos se trabaron en `:-` (leerlo como “sí”). Reforzar en B0. - El ejemplo `ancestro/2` fue exitoso → usarlo de referencia constante. - Tiempos de pizarrón se subestimaron → agregar 3 min buffer por bloque de trazado.

De cursadas anteriores (memoria colectiva `_edu-memory/memory.db`): - El corte (`!`) siempre confunde. Reforzar con F-061 (diagrama de barrera). - La trampa de `\+` con variables libres aparece en el 40% de los parciales. Insistir en B5. - CLP(FD) despierta mucho interés → tenerlo como “extensión premium” en B11.

11. Preparación del docente sobre el tema

Lecturas recomendadas (ordenadas por prioridad)

1. **Sebesta cap. 16** (básico)
2. **Triska — The Power of Prolog** (moderno, online)
3. **Sterling & Shapiro cap. 3, 6, 8** (avanzado)
4. **Gabbrielli & Martini cap. Logic Programming** (formal)
5. **Bratko — Prolog Programming for AI** (aplicaciones)

Videos recomendados

- Markus Triska, “The Power of Prolog” (series YouTube)
- Peter Norvig, “Symbolic AI Reloaded” (2023)

Papers 2024–2026

- DeepProbLog 2.0 (Manhaeve et al., 2024)
 - AlphaProof technical report (DeepMind, 2024)
 - “LLMs as Logic Engines” (Yao et al., 2025)
-

12. Siguientes temas en el ciclo

- **Tema 08:** Paradigma OO con TypeScript (empezamos el módulo IV).
- **Tema 09:** Variables, binding y ámbito (módulo VI).

Conexión narrativa para el próximo: “Ahora que vieron **paradigmas** (imperativo, funcional, lógico, OO), vamos a meternos en los **fundamentos** de cualquier lenguaje”.

Guía del profesor — Paradigmas y Lenguajes de Programación 2026 — UNTdF — 2026-04-21

Autocontenida: todo lo que necesitás para dictar el tema está acá o referenciado.